CS3212 Algorithms

Aya Zirikly
10/29/2026

Introduction to Depth-First Search (DFS)

Deep-First Traversal: DFS explores a graph by diving as deep as possible along each branch before backtracking.

Exploration Strategy: From a starting vertex, DFS visits an unvisited neighbor and recursively continues along that path until it reaches a node with no unvisited neighbors (a dead end), then backtracks.

Data Structure: Uses a Last-In, First-Out (LIFO) Stack, either explicitly or implicitly via the system call stack (recursion).

CLRS 3-Coloring Model

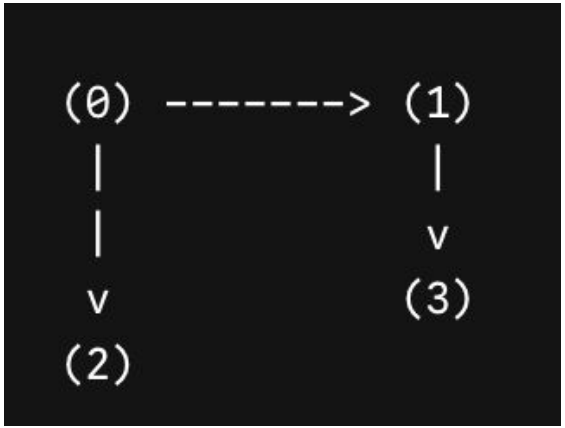
To track state during exploration, the CLRS algorithm assigns each vertex a color:

- **WHITE:** Unvisited vertex.
- **GRAY:** Discovered vertex, currently being explored (on the call stack).
- **BLACK:** Finished vertex; all outgoing edges and reachable descendants have been fully explored.

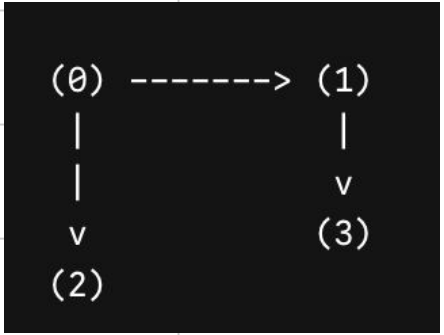
DFS records two timestamps for each vertex `u`:

- `u.d` **(Discovery Time):** The step counter when `u` is first discovered (turns GRAY).
- `u.f` **(Finishing Time):** The step counter when `u` finishes exploration (turns BLACK).
- **Parent Pointer (`u.pi`):** Points to the vertex that discovered `u`.

Step-by-Step Traversal Example



Step	Action / Event	Active Call Stack	Vertex Color States	Timestamps [d / f]
1	Discover vertex 0	[0]	0: GRAY	0.d = 1
2	Visit neighbor 1 from 0	[0, 1]	0: GRAY, 1: GRAY	1.d = 2
3	Visit neighbor 3 from 1	[0, 1, 3]	0: GRAY, 1: GRAY, 3: GRAY	3.d = 3
4	3 has no outgoing edges	[0, 1]	3: BLACK	3.f = 4
5	1 has no more neighbors	[0]	1: BLACK	1.f = 5
6	Visit neighbor 2 from 0	[0, 2]	2: GRAY	2.d = 6
7	2 has no outgoing edges	[0]	2: BLACK	2.f = 7
8	0 has no more neighbors	[]	0: BLACK	0.f = 8



Final Timestamp Output

- Vertex 0: `[1 / 8]`
- Vertex 1: `[2 / 5]`
- Vertex 3: `[3 / 4]`
- Vertex 2: `[6 / 7]`

Edge Types

Tree edge



A **Tree Edge** is an edge `u -> v` encountered when node `v` is discovered for the first time. Exploring this edge adds `v` to the DFS search tree.

Node `v` is **WHITE** when edge `u -> v` is explored.

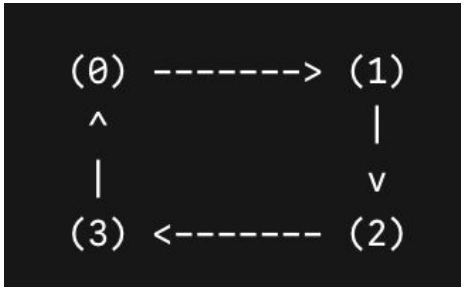
Step-by-Step Trace

1. DFS starts at **0** (`0.d = 1`, `0` turns **GRAY**).
2. From node 0, DFS examines edge **0 -> 1**.
3. Node **1** is **WHITE** (unvisited).
4. Edge **(0 -> 1)** is a **Tree Edge**. Node 1 is now discovered (`1.d = 2`, `1` turns **GRAY**).

Back edge

A **Back Edge** is an edge `u -> v` pointing from node `u` back to an ancestor node `v` that is currently active on the recursion call stack.

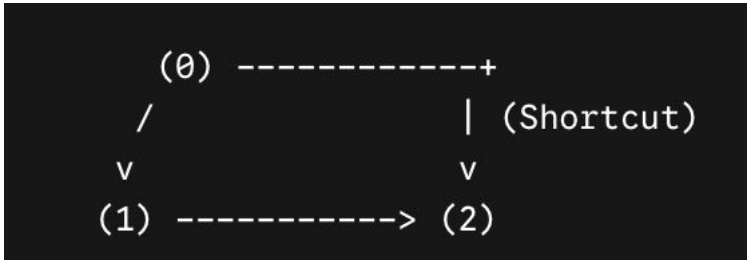
Node `v` is **GRAY** when edge `u -> v` is explored.
Key Significance: Discovering a Back Edge proves the graph contains a **directed cycle**.



Step-by-Step Trace

1. DFS starts at **0** (`0.d = 1`, active on stack, **GRAY**).
2. DFS visits neighbor **1** (`1.d = 2`, active on stack, **GRAY**).
3. DFS visits neighbor **2** (`2.d = 3`, active on stack, **GRAY**).
4. DFS visits neighbor **3** (`3.d = 4`, active on stack, **GRAY**).
5. At node 3, DFS examines outgoing edge **3 -> 0**.
6. Node **0** is currently **GRAY** (it is an ancestor sitting on the call stack).
7. Edge **(3 -> 0)** is a **Back Edge** (completing cycle `0 -> 1 -> 2 -> 3 -> 0`).

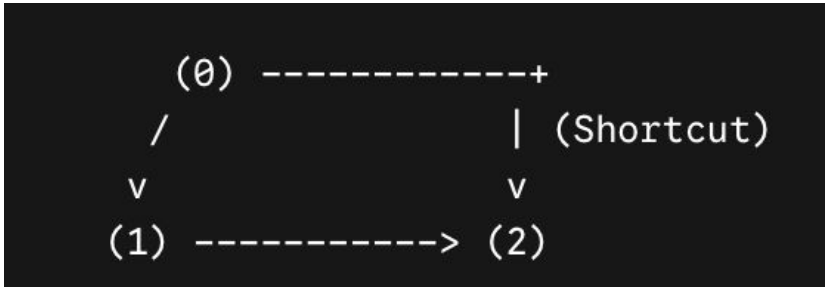
Forward edge



A **Forward Edge** is a non-tree edge `u -> v` pointing from an ancestor `u` to a descendant `v` that was already fully explored and finished via a deeper path.

Node `v` is **BLACK** when edge `u -> v` is explored, and **`u.d < v.d`** (node `u` was discovered before node `v`).

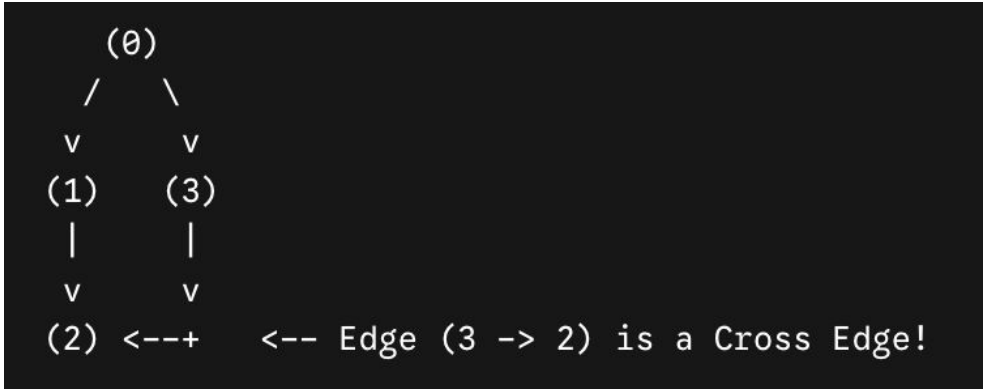
Forward edge



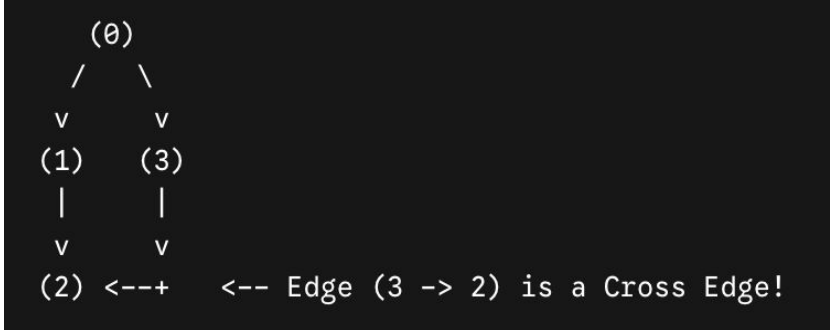
Step-by-Step Trace

1. DFS starts at **0** (`0.d = 1`, **GRAY**).
2. DFS explores tree edge **0 -> 1** (`1.d = 2`, **GRAY**).
3. From node 1, DFS explores tree edge **1 -> 2** (`2.d = 3`, **GRAY**).
4. Node 2 has no outgoing edges: it finishes (`2.f = 4`, turns **BLACK**).
5. Node 1 has no more outgoing edges: it finishes (`1.f = 5`, turns **BLACK**).
6. DFS backtracks to node 0 (`0.d = 1`, still **GRAY**).
7. Node 0 now examines its second outgoing edge: **0 -> 2**.
8. Node 2 is already **BLACK** (finished).
9. Compare discovery times: `0.d = 1` and `2.d = 3` (`0.d < 2.d`).
10. Edge **(0 -> 2)** is a **Forward Edge**—it shortcuts directly from ancestor 0 down to descendant 2, which was already processed via path `0 -> 1 -> 2`.

Cross edge



A **Cross Edge** is a directed edge `u -> v` pointing to a node `v` that belongs to a completely separate branch or depth-first tree, where no ancestor-descendant relationship exists between `u` and `v`.



DFS Execution Order:

1. DFS starts at **0** (`0.d = 1`).
2. DFS explores down the left branch to **1** (`1.d = 2`).
3. DFS continues down to **2** (`2.d = 3`).
4. Node **2** has no outgoing edges, so it finishes and turns **BLACK** (`2.f = 4`).
5. Node **1** finishes and turns **BLACK** (`1.f = 5`).
6. DFS backtracks to **0** and starts down the right branch to **3** (`3.d = 6`).
7. At node **3**, DFS examines the edge **3 -> 2**.

Evaluating Edge (3 -> 2):

- Node 2 was discovered at time 3 (`2.d = 3`).
- Node 3 was discovered at time 6 (`3.d = 6`).
- Node 2 is already **BLACK** (finished).
- Because `3.d > 2.d` (6 > 3) and node 2 is **BLACK**, edge `3 -> 2` is a **Cross Edge**. It "crosses" over from the right branch to an already-finished left branch.

Summary

Edge Type	Status of Node v when u -> v is Examined	Timestamp Relation	Graph Structure Meaning
Tree Edge	WHITE	N/A	First time discovering v (parent-child relationship).
Back Edge	GRAY	u.d > v.d	Points to active ancestor on stack (indicates a cycle).
Forward Edge	BLACK	u.d < v.d	Non-tree edge pointing from ancestor to a descendant visited earlier.
Cross Edge	BLACK	u.d > v.d	Points across to a completely separate branch/tree visited earlier.

Complexity Analysis

Metric	Adjacency List	Adjacency Matrix	Explanation
Time Complexity	O(V + E)	O(V^2)	DFS-VISIT is called once per vertex V. The inner loop examines all adjacency list entries (total E edges). Matrix requires scanning all V entries per vertex.
Space Complexity	O(V)	O(V)	Storage needed for recursion stack depth, color arrays, pi, d, and f timestamps.

Space Complexity Breakdown

- **Worst-Case Stack Memory:** O(V) in a long chain graph (e.g., 0 -> 1 -> 2 -> 3).
- **Best-Case Stack Memory:** O(1) if the graph has no edges.

Key Performance Notes

- **Optimal Representation:** Adjacency List provides true linear time traversal O(V + E).
- **Graph Density Impact:** In sparse graphs where E << V^2, Adjacency Lists offer significant speedups over Adjacency Matrices.

Algorithm Comparison Summary (BFS vs. DFS)

Feature	Breadth-First Search (BFS)	Depth-First Search (DFS)
Primary Data Structure	FIFO Queue	LIFO Stack (or Recursion Stack)
Exploration Pattern	Radiates outward level-by-level	Dives as deep as possible, then backtracks
Vertex Annotations (CLRS)	Distance dist[], Parent pi[]	Colors (WHITE/GRAY/BLACK), Timestamps u.d, u.f, Parent pi[]
Shortest Path Property	Guarantees shortest path in unweighted graphs	Does NOT guarantee shortest path
Memory Bottleneck	Proportional to maximum graph width: O(W)	Proportional to maximum graph depth: O(H)

Algorithm Comparison Summary (BFS vs. DFS)

Feature	Breadth-First Search (BFS)	Depth-First Search (DFS)
Adjacency List Time	O(V + E)	O(V + E)
Adjacency Matrix Time	O(V^2)	O(V^2)
Edge Classification	N/A	Tree, Back, Forward, Cross Edges
Cycle Detection	Track visited nodes	Look for Back Edges (pointing to GRAY nodes)

Topological Sorting for ordering tasks with dependencies in Directed Acyclic Graphs (DAGs)

Introduction to Topological Sorting & DAGs

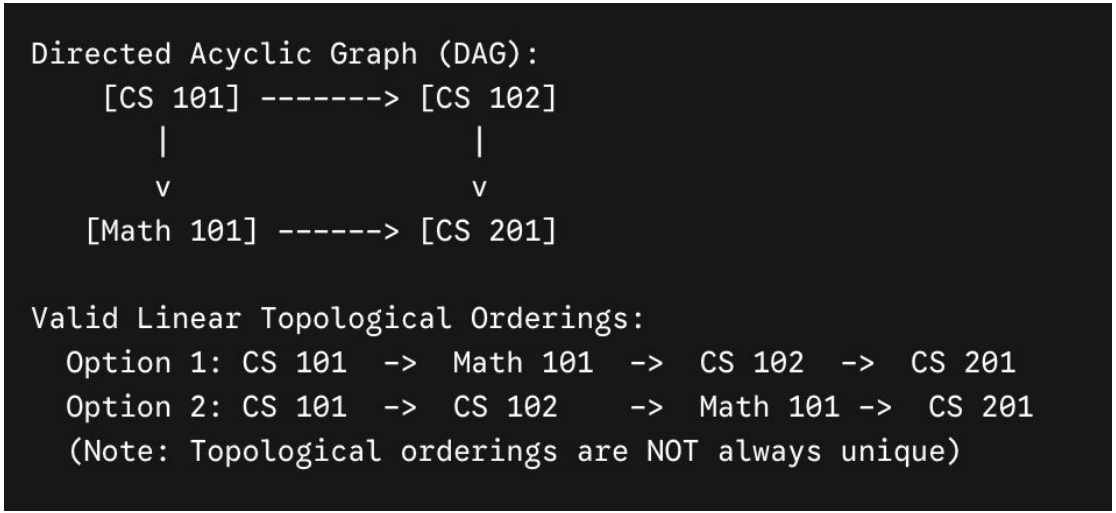
- **Directed Acyclic Graph (DAG):** A directed graph containing no directed cycles.
- **Precedence Constraints:** A directed edge u -> v represents a mandatory ordering constraint where task u must complete before task v can begin.
- **Source & Sink Guarantee:** Every finite DAG contains at least one **source** vertex (in-degree = 0) and at least one **sink** vertex (out-degree = 0).

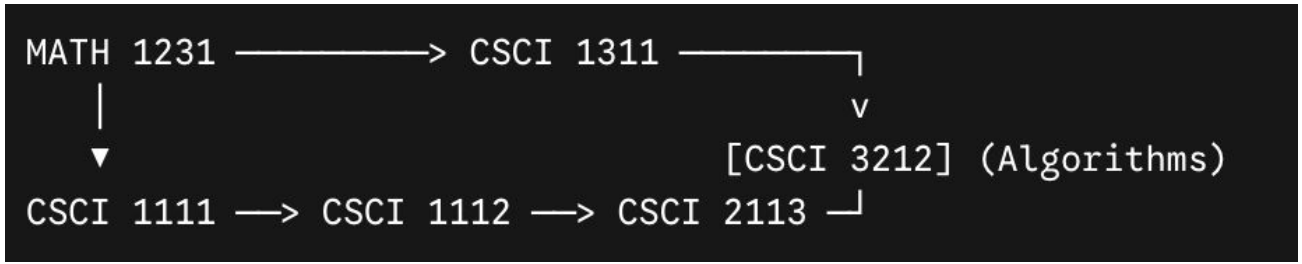
Topological Sorting for ordering tasks with dependencies in Directed Acyclic Graphs (DAGs)

Partial Ordering: Unlike a linear array (total order), a DAG represents a partial order—tasks on independent branches can be executed in parallel.

Total Order vs. Partial Order

- **Total Order (Linear Array):** Every single pair of elements has a fixed relative position.
  - *Example:* In an array [A, B, C, D], A must come before B, B before C, and C before D. There is zero ambiguity about which item comes first for any pair you choose.
- **Partial Order (DAG):** Only **some** pairs of vertices have a directed path forcing an ordering between them.
  - If an edge exists A \to B, then A **must** happen before B.
  - If two tasks C and D lie on separate branches with no path between them, neither task depends on the other. Their relative order is **unspecified**.

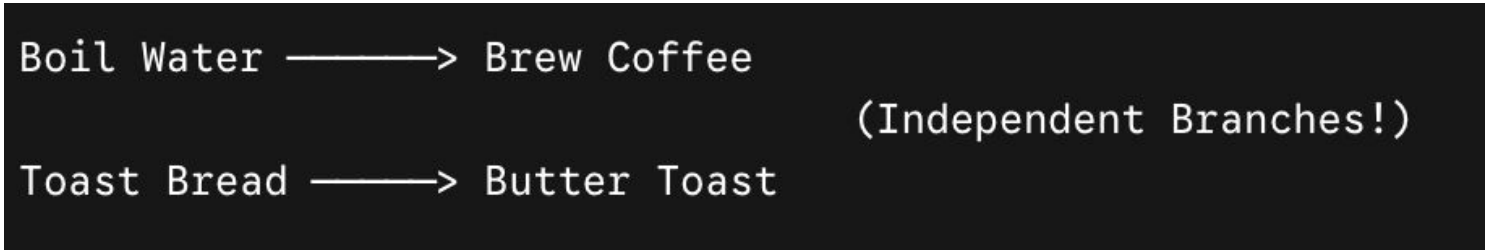

Directed Acyclic Graph (DAG):
[CS 101] -------> [CS 102]
[Math 101] ------> [CS 201]
Valid Linear Topological Orderings:
Option 1: CS 101 -> Math 101 -> CS 102 -> CS 201
Option 2: CS 101 -> CS 102 -> Math 101 -> CS 201
(Note: Topological orderings are NOT always unique)


MATH 1231
CSCI 1311
[CSCI 3212] (Algorithms)
CSCI 1111 —> CSCI 1112 —> CSCI 2113

Why Independent Branches Can Run in Parallel

Because tasks C and D have no direct or indirect dependency on one another, a task manager or compiler does not need to wait for C to finish before starting D.

- **Parallel Execution:** You can run Task C on CPU Core 1 and Task D on CPU Core 2 at the exact same time.
- **Flexible Order:** If running sequentially on a single thread, you can execute C then D, or D then C. Both choices produce a valid outcome.



Fixed Dependencies (Partial Rules):

- You must *Boil Water* before you can *Brew Coffee*.
- You must *Toast Bread* before you can *Butter Toast*.

Independent Pairs (No Constraint):

- *Boiling Water* and *Toasting Bread* have no arrow between them.

Execution: You can boil water and toast bread **in parallel** on two separate appliances.

Topological Sorting via Kahn's Algorithm

In-Degree BFS Approach

- **Zero In-Degree Principle:** Any vertex with in-degree = 0 has zero remaining prerequisites and is ready for immediate execution.
- **Greedy Elimination:** Process a 0-in-degree vertex, append it to the sorted schedule, and decrement the in-degree of all its adjacent neighbors.

BFS Topological Sort (Kahn's In-Degree Algorithm)

How It Works

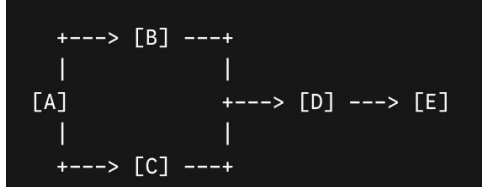
1. Compute the in-degree (number of incoming edges) for every vertex.
2. Initialize a Queue with all vertices that have an in-degree of 0 (tasks with no prerequisites).
3. Dequeue a vertex u, append it to the topological output list, and decrement the in-degree of all its outgoing neighbors v.
4. If a neighbor's in-degree drops to 0, enqueue it. Repeat until the queue is empty.



Vertices: `A`, `B`, `C`, `D`, `E`
Edges: `A -> B`, `A -> C`, `B -> D`, `C -> D`, `D -> E`

Initial In-Degree Array

- `in_degree[A] = 0`
- `in_degree[B] = 1` (from A)
- `in_degree[C] = 1` (from A)
- `in_degree[D] = 2` (from B, C)
- `in_degree[E] = 1` (from D)



Step	Queue at Start	Dequeued Node u	In-Degree Updates	Enqueued Nodes	Queue After Enqueue	Output List (topo_order)
0	[]	-	A:0, B:1, C:1, D:2, E:1	A	[A]	[]
1	[A]	A	Decrement B (1 -> 0)Decrement C (1 -> 0)	B, C	[B, C]	[A]
2	[B, C]	B	Decrement D (2 -> 1)	None	[C]	[A, B]
3	[C]	C	Decrement D (1 -> 0)	D	[D]	[A, B, C]
4	[D]	D	Decrement E (1 -> 0)	E	[E]	[A, B, C, D]
5	[E]	E	None	None	[]	[A, B, C, D, E]

```
FUNCTION Kahns_Topological_Sort(Graph):
    Compute in_degree for all vertices
    Q = Queue containing all vertices with in_degree == 0
    topo_order = []

    WHILE Q is not empty:
        u = DEQUEUE(Q)
        APPEND u TO topo_order

        FOR EACH neighbor v OF u:
            in_degree[v] = in_degree[v] - 1
            IF in_degree[v] == 0:
                ENQUEUE(Q, v)

    IF length(topo_order) == total_vertices:
        RETURN topo_order
    ELSE:
        RETURN "Error: Cycle Detected"
```

DFS Topological Sort & Cycle Detection

DFS Approach (Finishing Time Reversal)

- **Core Theory:** Running Depth-First Search on a DAG guarantees that for every directed edge u -> v, the DFS finishing time of u (u.f) is strictly greater than the finishing time of v (v.f).
- **Sorting Rule:** Order vertices by **decreasing finishing time** (u.f) by prepending each node to a linked list as it turns `BLACK`.

DFS Topological Sort

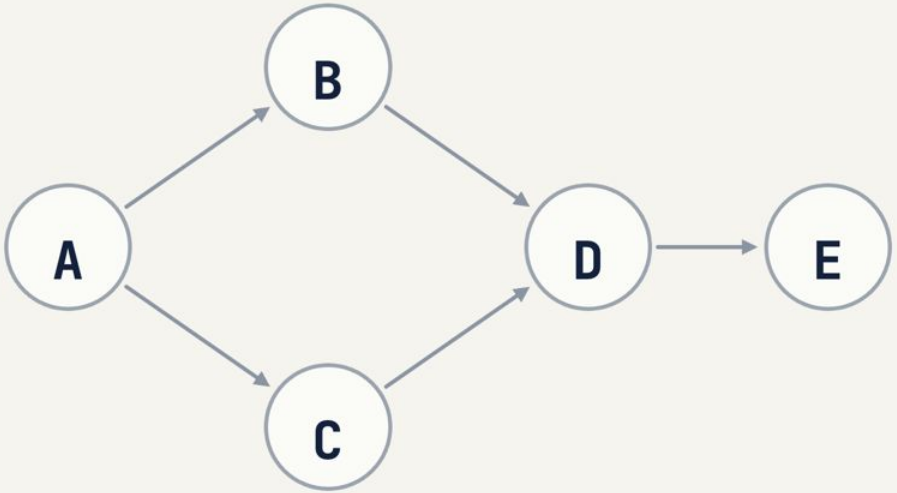
How It Works

1. Run standard Depth-First Search on the graph.
2. Record discovery times (`u.d`) when a vertex turns `GRAY`, and finishing times (`u.f`) when a vertex turns `BLACK` (all descendants fully explored).
3. Whenever a vertex turns `BLACK`, prepend it to the front of a linked list.
4. The final list gives the vertices sorted by **decreasing order of finishing time**.

```
DFS-TOPOLOGICAL-SORT(Graph):
    Run DFS(Graph) to compute finishing times u.f for all u
    As each vertex u completes DFS-VISIT (turns BLACK):
        PREPEND u TO topological_list
    RETURN topological_list
```

THE INPUT

A directed acyclic graph



EDGES

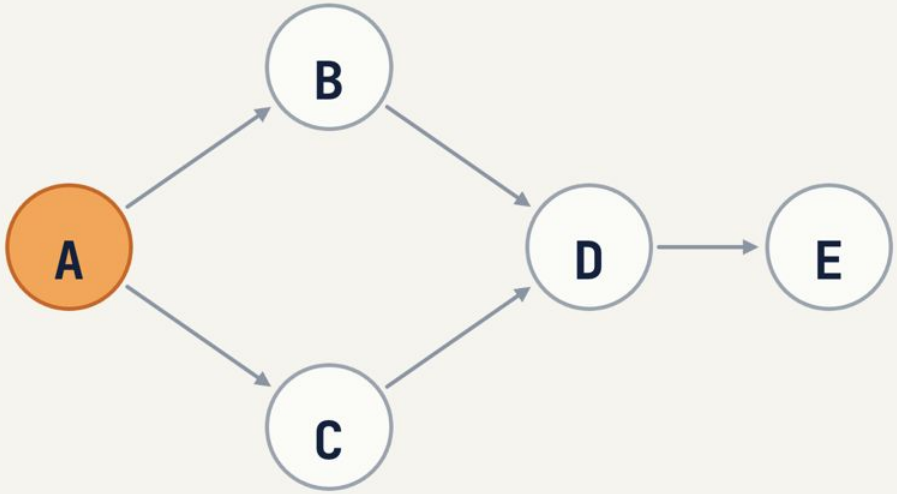
A → B
A → C
B → D
C → D
D → E

Goal: list every node so that each edge points from an earlier node to a later one.

Every node starts unvisited. The call stack and the output list are both empty.

STEP 1 OF 10

Visit A



CALL STACK

A

OUTPUT (FRONT FIRST)

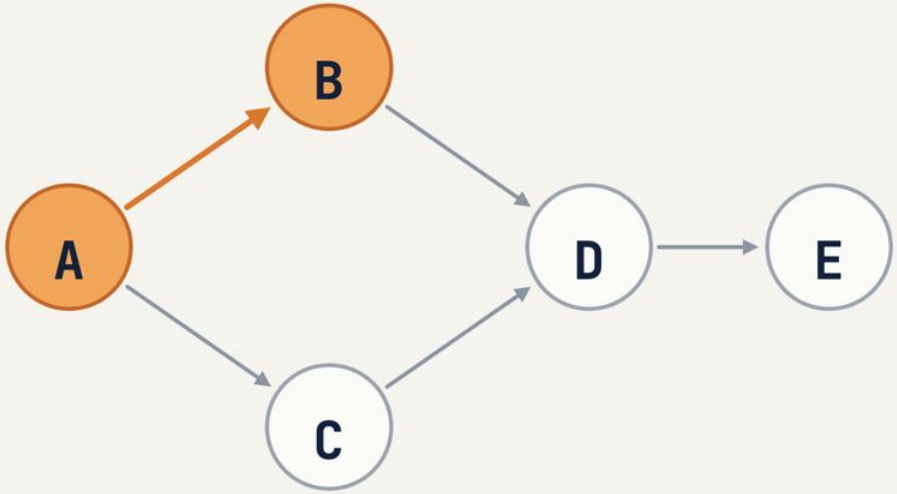
empty

- Unvisited
- In progress (on the stack)
- Finished

Start at A, the first unvisited node. Mark it in progress and push it on the stack.

STEP 2 OF 10

Visit B



CALL STACK

A B

OUTPUT (FRONT FIRST)

empty

Unvisited

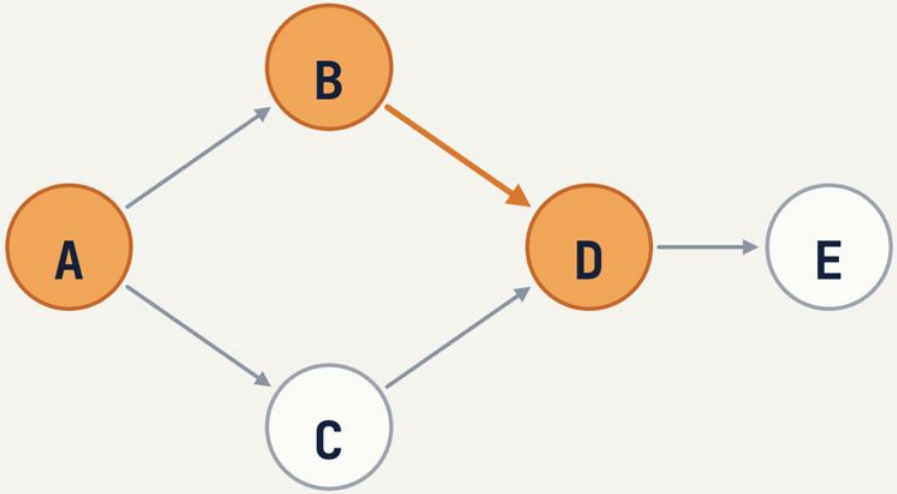
In progress (on the stack)

Finished

A → B: A's first neighbor is B, which is unvisited. Recurse into B.

STEP 3 OF 10

Visit D



B → D: D is unvisited, so recurse into D.

CALL STACK

A B D

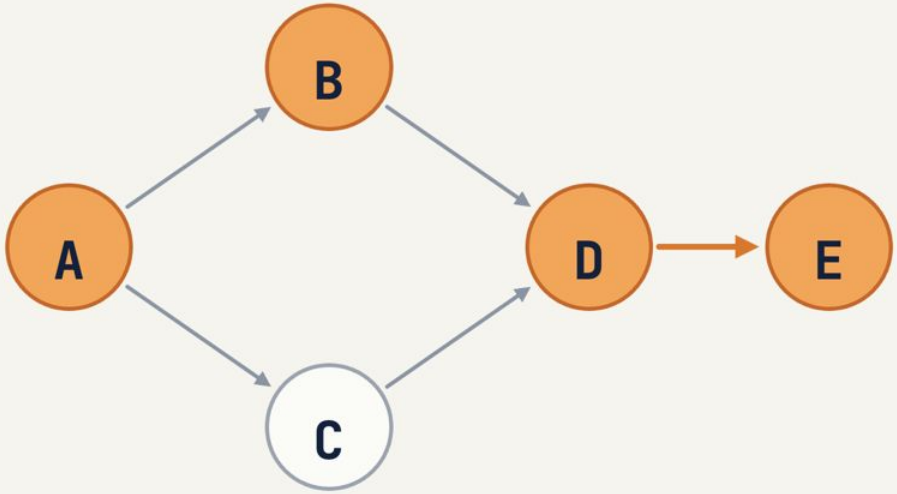
OUTPUT (FRONT FIRST)

empty

Unvisited
In progress (on the stack)
Finished

STEP 4 OF 10

Visit E



D → E: E is unvisited, so recurse into E.

CALL STACK

A B D E

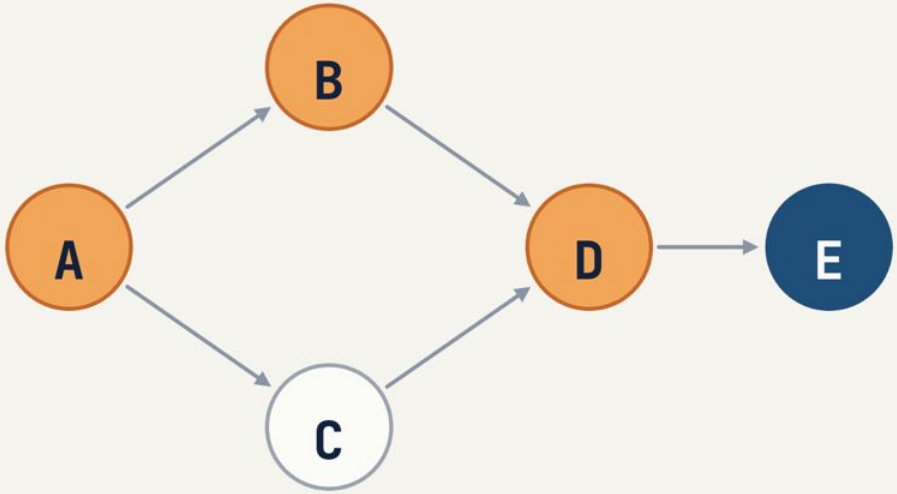
OUTPUT (FRONT FIRST)

empty

Unvisited
In progress (on the stack)
Finished

STEP 5 OF 10

Finish E



CALL STACK

A B D

OUTPUT (FRONT FIRST)

E

Unvisited

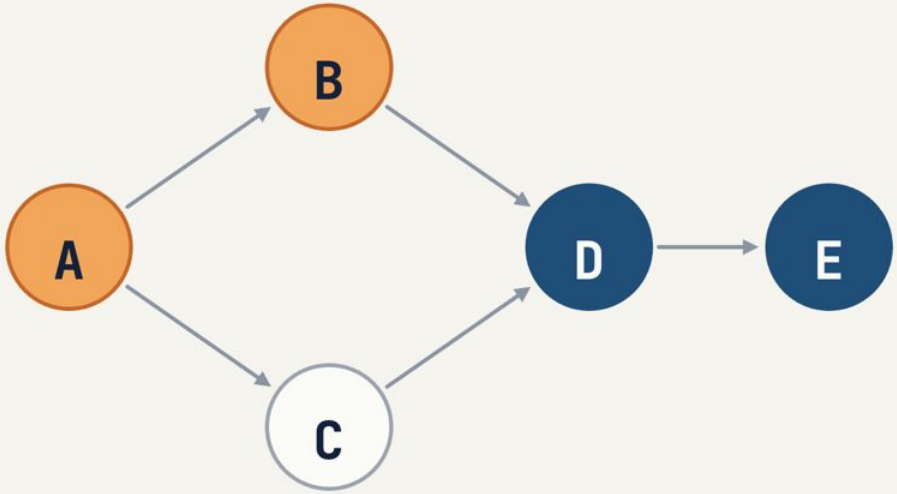
In progress (on the stack)

Finished

E has no outgoing edges. Finish E: pop it off the stack and put it at the front of the output.

STEP 6 OF 10

Finish D



CALL STACK



OUTPUT (FRONT FIRST)

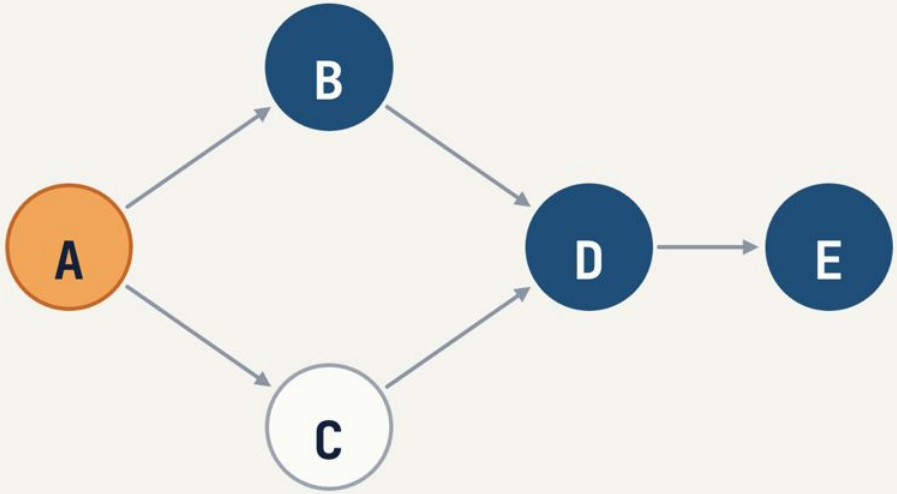


- Unvisited
- In progress (on the stack)
- Finished

Back in D: no neighbors left. Finish D; it goes to the front, ahead of E.

STEP 7 OF 10

Finish B



CALL STACK

A

OUTPUT (FRONT FIRST)

B D E

Unvisited

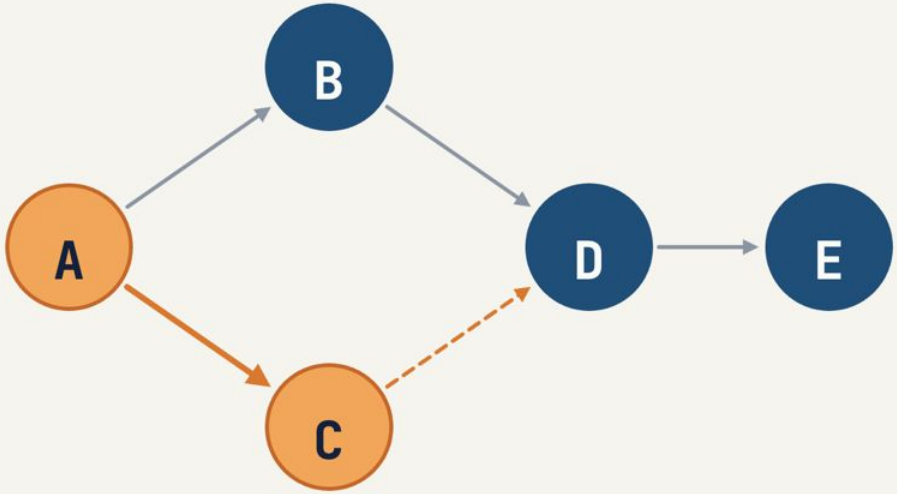
In progress (on the stack)

Finished

Back in B: its only neighbor, D, is done. Finish B; it goes to the front.

STEP 8 OF 10

Visit C, skip D



CALL STACK



OUTPUT (FRONT FIRST)



Unvisited

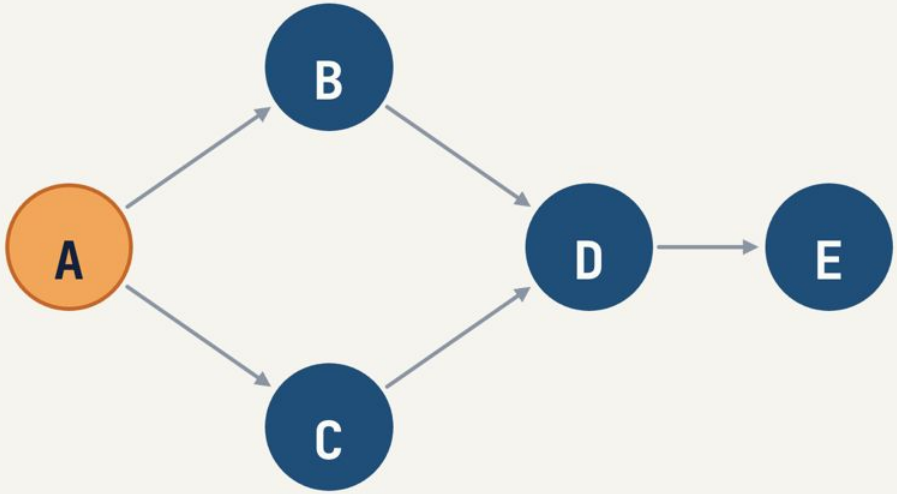
In progress (on the stack)

Finished

A → C: C is unvisited, so recurse. Then C → D (dashed) is skipped: D is already finished.

STEP 9 OF 10

Finish C



CALL STACK



OUTPUT (FRONT FIRST)



Unvisited

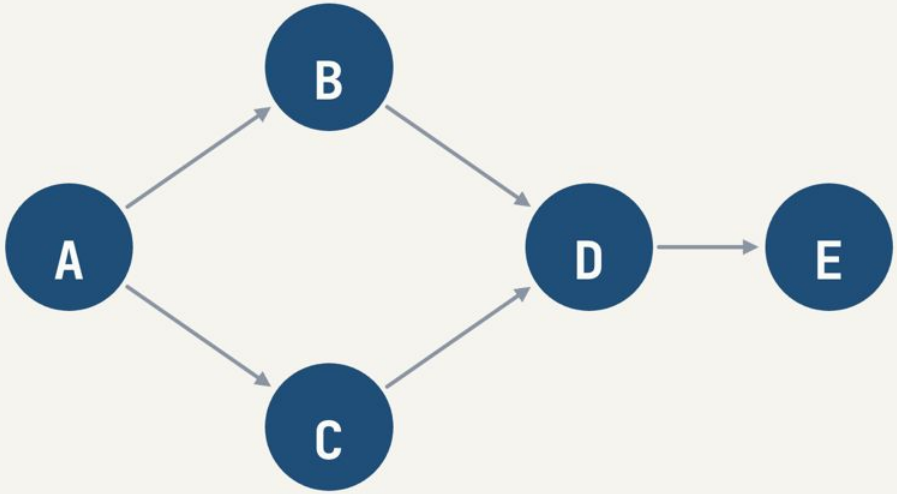
In progress (on the stack)

Finished

C has nothing left to explore. Finish C; it goes to the front, ahead of D.

STEP 10 OF 10

Finish A



CALL STACK

empty

OUTPUT (FRONT FIRST)

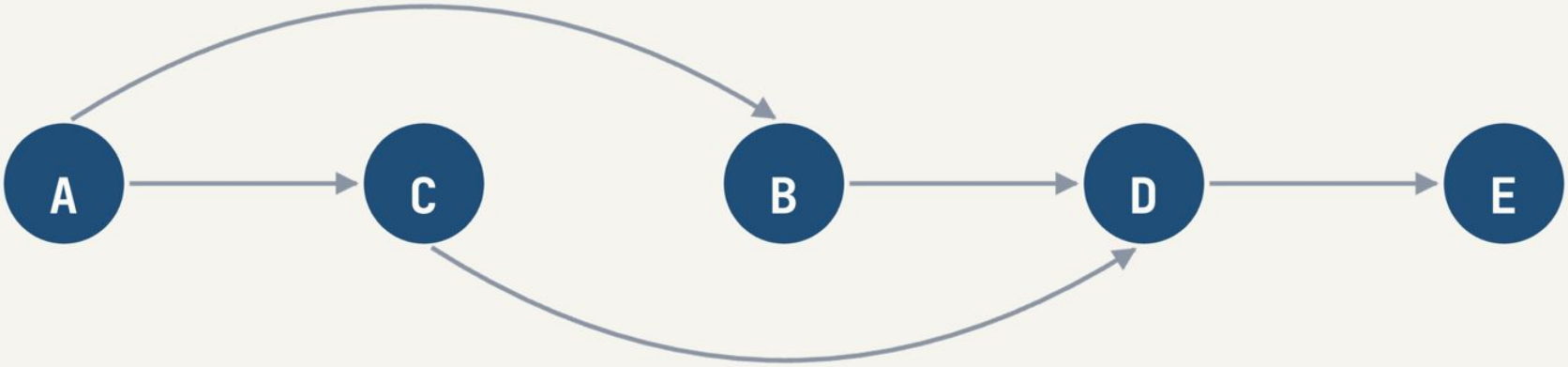
A C B D E

- Unvisited
- In progress (on the stack)
- Finished

All of A's neighbors are done. Finish A. B–E are already visited, so the outer loop ends.

RESULT

Topological order: A, C, B, D, E



Every edge now points left to right. Same list as the finish order E, D, B, C, A, read backwards.

WHY IT WORKS

Three things to remember

Finish after descendants

A node finishes only after everything reachable from it has finished.

Prepend on finish

So each node lands in front of all its descendants, and every edge points forward.

Linear time

Each node and each edge is handled once: O(V + E).

Orders aren't unique: A, B, C, D, E is also valid. Which one DFS returns depends on the order neighbors are explored.

Cycle Detection Mechanisms

- **DFS Back-Edge Check:** A directed graph contains a cycle if and only if DFS encounters an edge u -> v where v is currently `GRAY` (active on the recursion stack).
- **Kahn's Count Check:** If Kahn's algorithm terminates and length(topo_order) < total vertices V, the remaining unprocessed nodes form a cycle.

Kahn's algorithm

THE IDEA

Peel off in-degree-0 nodes, then count them

```
indeg[v] = edges pointing into v
queue = every v with indeg 0
count = 0
while queue not empty:
  u = queue.pop()
  output u; count += 1
  for each edge u → v:
    indeg[v] -= 1
    if indeg[v] == 0: queue.push(v)
if count < V: cycle!
```

In-degree

The dark badge above each node: how many unprocessed edges still point into it.

Processing

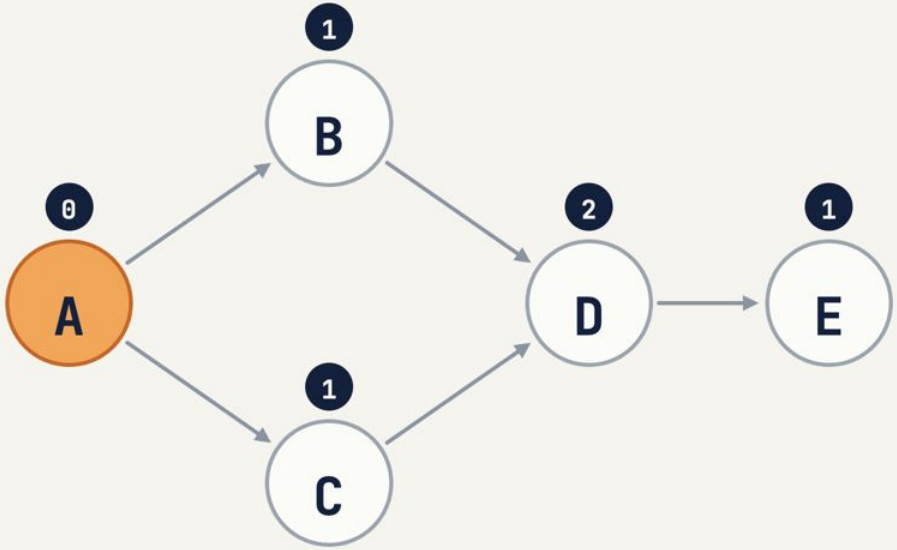
Output the node and delete its outgoing edges. Any neighbor that drops to 0 joins the queue.

The count check

When the queue runs dry, compare count with V. Fewer means some nodes could never reach 0: a cycle.

RUN 1 · THE ORIGINAL GRAPH

Start: queue every in-degree-0 node



QUEUE (IN-DEGREE 0)

A

OUTPUT

empty

PROCESSED COUNT

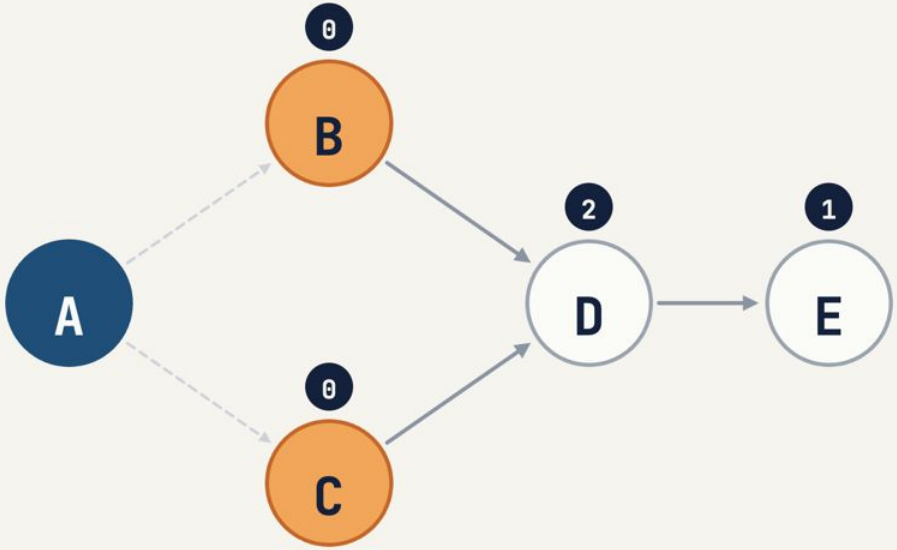
0 / 5 nodes

Badges show in-degrees. Only A has none pointing in, so the queue starts as [A].

Waiting · Ready · Output · Stuck

RUN 1 · STEP 1 OF 5

Process A



QUEUE (IN-DEGREE 0)

B C

OUTPUT

A

PROCESSED COUNT

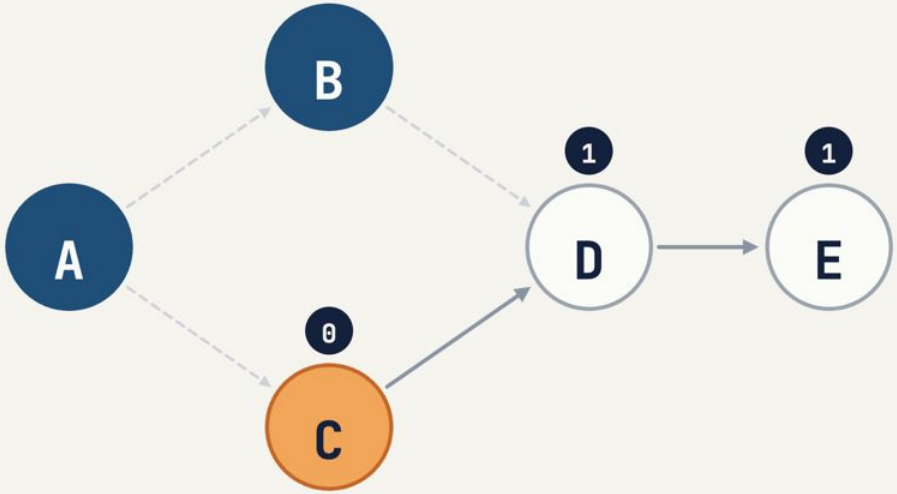
1 / 5 nodes

Output A and delete A → B and A → C (dashed). B and C both drop to 0 and join the queue.

Waiting Ready Output Stuck

RUN 1 · STEP 2 OF 5

Process B



QUEUE (IN-DEGREE 0)

C

OUTPUT

A B

PROCESSED COUNT

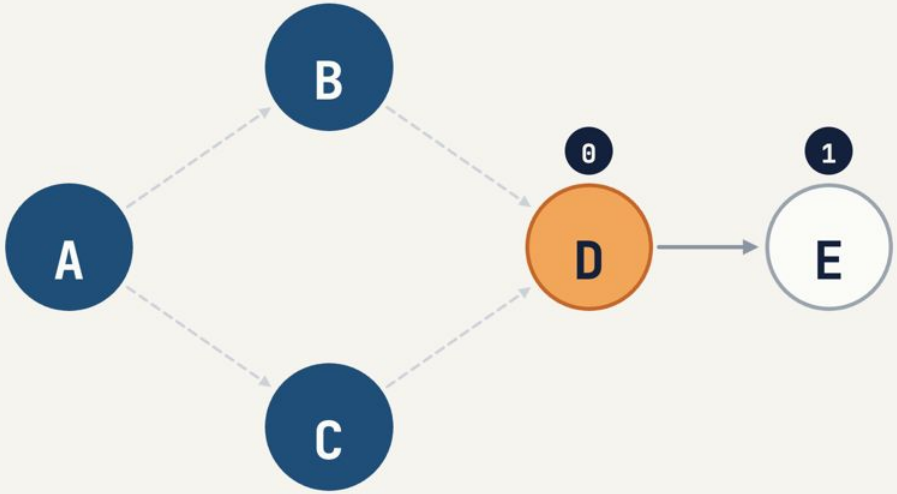
2 / 5 nodes

Output B and delete B → D. D drops from 2 to 1: it still waits on C.

Waiting Ready Output Stuck

RUN 1 · STEP 3 OF 5

Process C



QUEUE (IN-DEGREE 0)

D

OUTPUT

A B C

PROCESSED COUNT

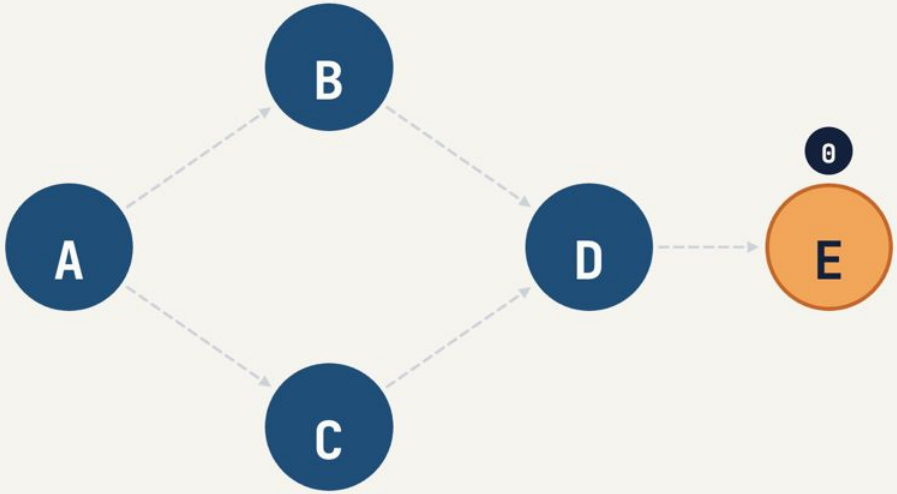
3 / 5 nodes

Output C and delete C → D. D finally reaches 0 and joins the queue.

Waiting Ready Output Stuck

RUN 1 · STEP 4 OF 5

Process D



QUEUE (IN-DEGREE 0)

E

OUTPUT

A B C D

PROCESSED COUNT

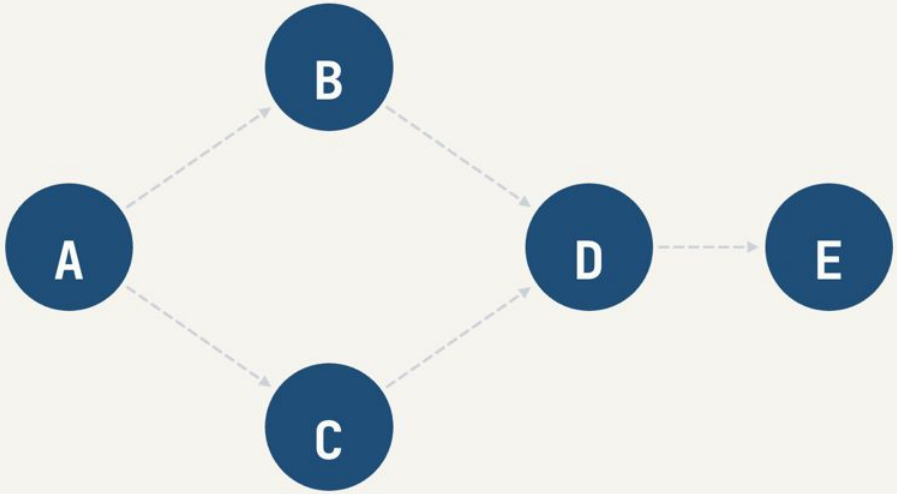
4 / 5 nodes

Output D and delete D → E. E drops to 0 and joins the queue.

Waiting Ready Output Stuck

RUN 1 · STEP 5 OF 5

Process E



QUEUE (IN-DEGREE 0)

empty

OUTPUT

A B C D E

PROCESSED COUNT

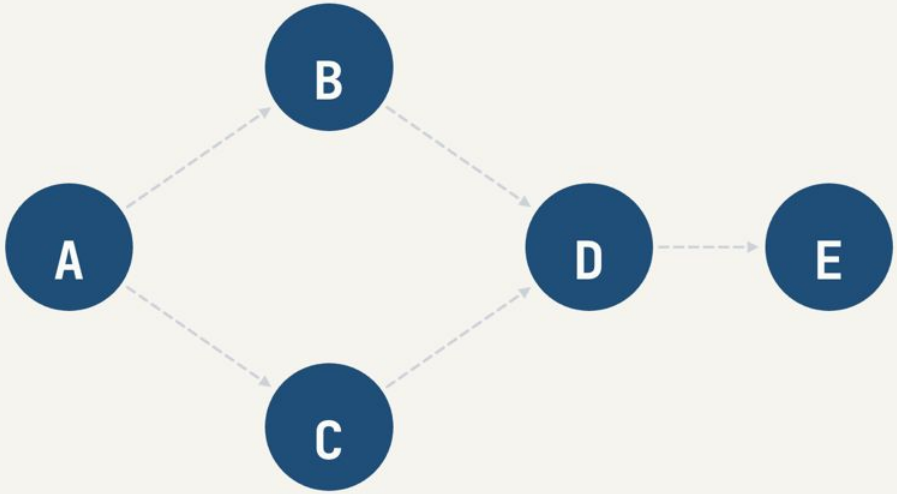
5 / 5 nodes

Output E. It has no outgoing edges, and the queue is now empty.

Waiting Ready Output Stuck

RUN 1 · COUNT CHECK

Every node made it out



The queue ran dry after processing all 5 nodes, so the graph is acyclic.

COUNT VS. V

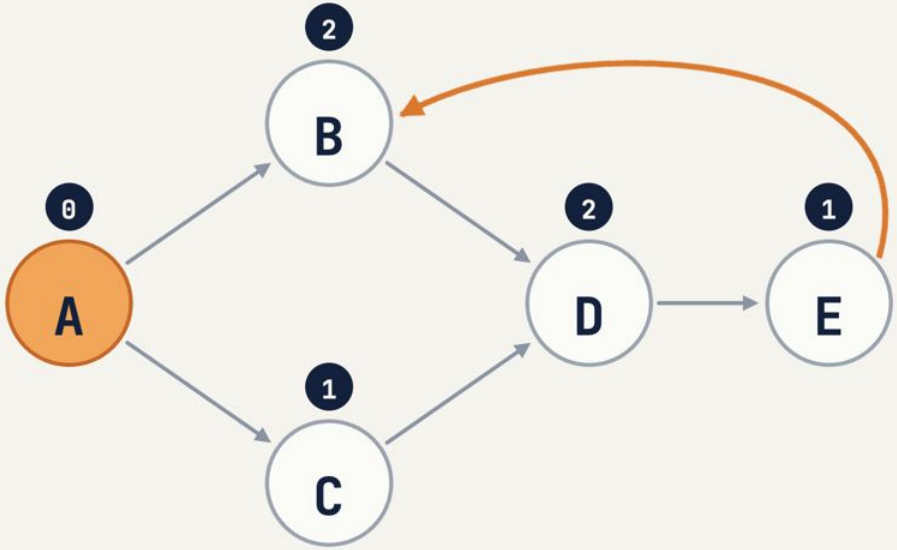
5 = 5

No cycle

Every node reached in-degree 0. The output A, B, C, D, E is a valid topological order.

RUN 2 · ADD ONE EDGE

Same graph, plus E → B



QUEUE (IN-DEGREE 0)

A

OUTPUT

empty

PROCESSED COUNT

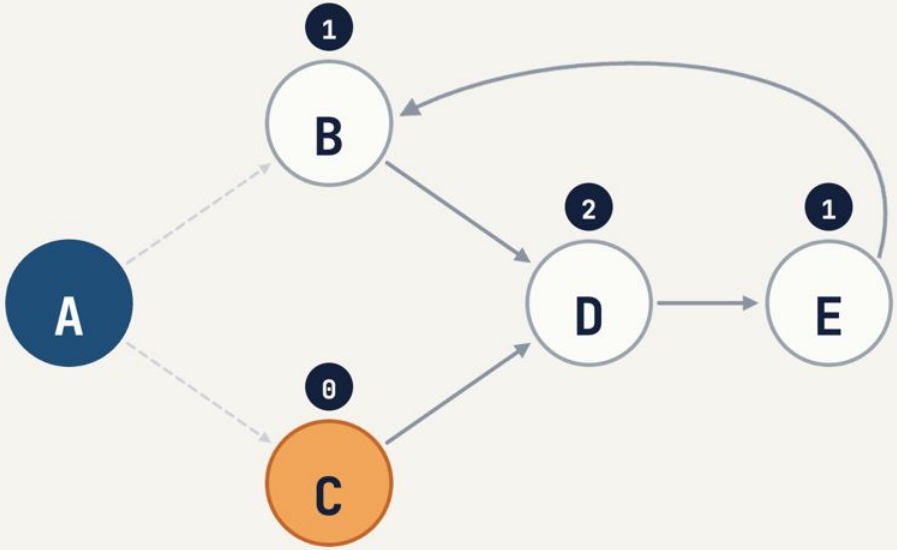
0 / 5 nodes

The new edge (orange) closes the loop B → D → E → B. B's in-degree rises to 2.

Waiting · Ready · Output · Stuck

RUN 2 · STEP 1

Process A



QUEUE (IN-DEGREE 0)

C

OUTPUT

A

PROCESSED COUNT

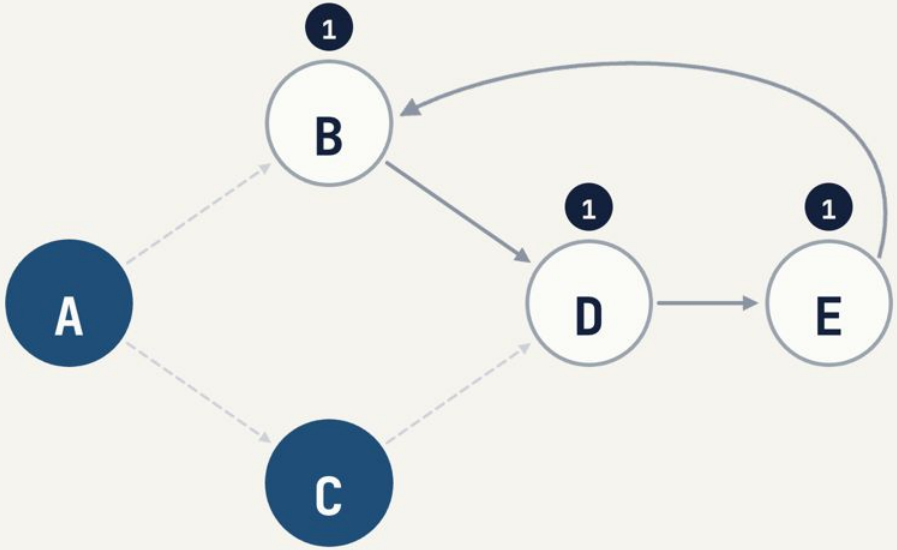
1 / 5 nodes

Output A, delete its edges. C reaches 0, but B only drops to 1: E → B still points in.

Waiting Ready Output Stuck

RUN 2 · STEP 2

Process C — then the queue runs dry



QUEUE (IN-DEGREE 0)

empty

OUTPUT

A C

PROCESSED COUNT

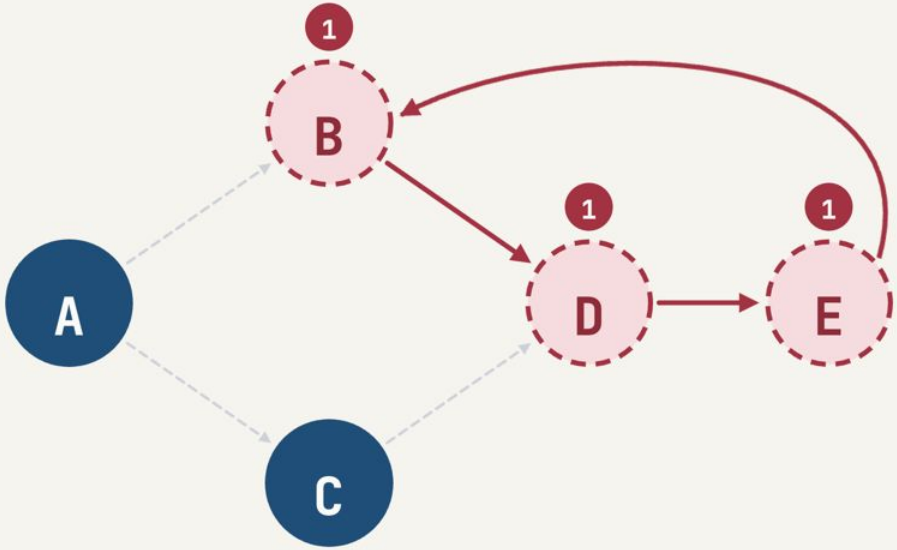
2 / 5 nodes

Output C, delete C → D. D drops to 1. No node is at 0, so the queue is empty and the loop stops.

Waiting Ready Output Stuck

RUN 2 · COUNT CHECK

Three nodes never made it out



The queue is empty with 3 nodes left. Their remaining edges (red) are exactly the cycle.

COUNT VS. V

2 < 5

Cycle detected

B, D and E are stuck at in-degree 1. Each waits on the one before it in the loop B → D → E → B.

WHY THE COUNT CHECK WORKS

A cycle can never reach in-degree 0

Always blocked

Every node on a cycle keeps an edge from the node before it, which is also stuck.

Never counted

Those nodes never enter the queue, so count stops short of V.

One comparison

count = V: valid order. count < V: cycle. Still O(V + E).

The leftover nodes include the cycle plus anything downstream of it, so they also tell you where to look.

Cycle Detection Mechanisms

DFS Back-Edge Check: A directed graph contains a cycle if and only if DFS encounters an edge u -> v where v is currently `GRAY` (active on the recursion stack).
Kahn's Count Check: If Kahn's algorithm terminates and length(topo_order) < total vertices V, the remaining unprocessed nodes form a cycle.

Key Takeaway

Topological orderings are **not unique**. Multiple valid orderings exist unless a single path visits every vertex in the graph (a Hamiltonian path).

Topological orderings are not unique unless…

1. Multiple Valid Orderings (The Standard Case)

Because a DAG represents a partial order, tasks on independent branches do not depend on each other. You can list those independent tasks in any relative order, creating multiple valid topological sequences.

2. The Unique Case: Directed Hamiltonian Path

A **Hamiltonian path** in a directed graph is a directed path that visits **every single vertex exactly once**.

If a DAG contains a directed Hamiltonian path, every node is bound to a strict chain relative to all other nodes. Because every node is tied into a single unbroken path, there are no independent choices left to swap—forcing **exactly one unique valid topological ordering**.

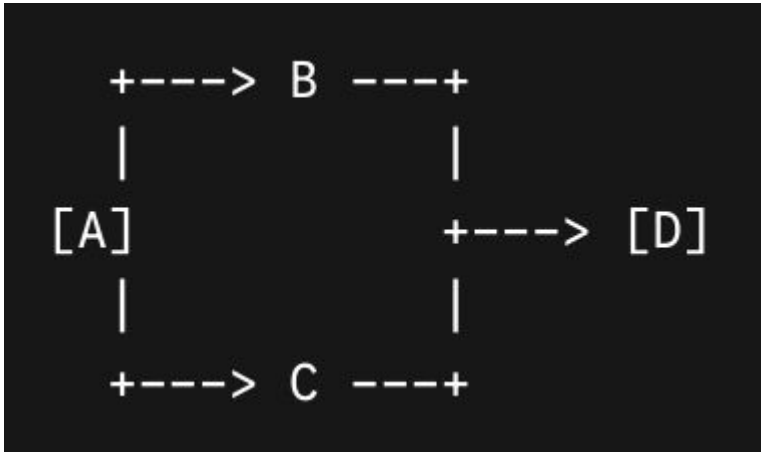
Multiple Orderings (No Hamiltonian Path)

Hamiltonian Path? No. You cannot visit A, B, C, and D in a single continuous path without backtracking.

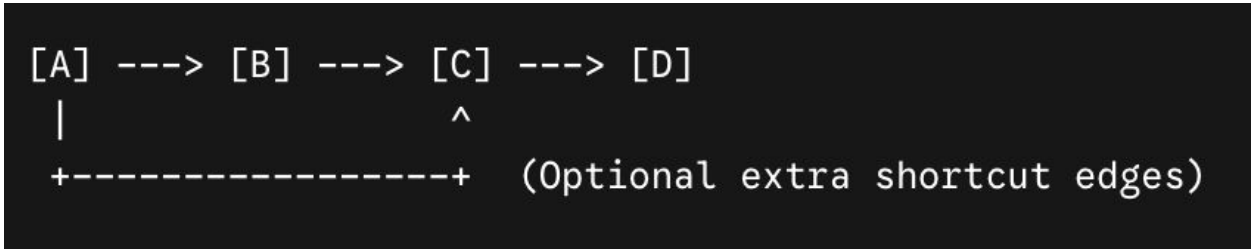
Valid Topological Orderings:

1. `A -> B -> C -> D`
2. `A -> C -> B -> D`

Outcome: Not unique (2 valid orders).



Unique Ordering (Contains a Hamiltonian Path)



Hamiltonian Path? Yes: `A -> B -> C -> D` visits every single node in a continuous line.
Valid Topological Orderings:

1. `A -> B -> C -> D`

Outcome: Unique (exactly 1 valid order).

O(V + E) Shortest Paths & Dynamic Programming

Linear-Time Single-Source Shortest Paths (SSSP)

- General Graphs vs. DAGs: Graphs with cycles cannot be topologically sorted, so you cannot know in advance which node's shortest path is finalized first.
- The DAG Advantage: By topologically sorting vertices first (via DFS or Kahn's BFS), you relax each outgoing edge in a single, left-to-right pass.
- Optimal Runtime: Every vertex and edge is examined exactly once, achieving O(V + E) linear time without needing complex priority structures.

```
DAG-SHORTEST-PATHS(Graph, source):
    topologically_sort(Graph)
    initialize dist[] = INFINITY for all vertices, dist[source] = 0

    FOR EACH vertex u IN topological_order:
        FOR EACH neighbor v OF u:
            RELAX edge (u, v):
                IF dist[v] > dist[u] + weight(u, v):
                    dist[v] = dist[u] + weight(u, v)
```

Why it is strictly O(V + E) in the Worst Case

The algorithm consists of three sequential steps, each with a deterministic worst-case bound:

1. **Topological Sort:** Sorting the DAG takes **O(V + E)** in the worst case using either DFS or Kahn's BFS algorithm.
2. **Initialization:** Setting distance values for all vertices takes **O(V)** time.
3. **Edge Relaxation:** Visiting every vertex once and relaxing all outgoing edges visits every node and examines every edge exactly once, taking **O(V + E)** time.

Combining these steps gives `O(V + E) + O(V) + O(V + E) = O(V + E)`.

Advantages & Benefits of DFS over BFS

Lower Memory Footprint for Deep Target Searches:

- DFS memory depends on the maximum tree depth: O(h).
- BFS memory depends on the maximum tree width: O(w).
- In wide graphs with deep branches, DFS uses significantly less memory than BFS.
- *Example:* **Chess or Sudoku Solvers.** A game tree has a huge branching factor. BFS would instantly run out of RAM trying to store every possible board state at level 5, while DFS safely explores one deep sequence of moves at a time.

Advantages & Benefits of DFS over BFS in Topological Sorting

Topological Sorting (DAGs):

- By sorting vertices by decreasing order of finishing times (`u.f`), DFS produces a valid topological ordering of a Directed Acyclic Graph in O(V + E) time.
- *Getting dressed* example: Ordering clothing items based on dependencies (e.g., socks must be put on before shoes, underwear before pants, shirt before tie).

Advantages & Benefits of DFS over BFS

Cycle Detection:

- Detecting a Back Edge during DFS instantly identifies directed cycles, which is cleaner than queue-based methods.
- *Example:* **Deadlock Detection & Circular Dependency Checks.** Detecting circular references in package managers (e.g., Package A needs Package B, which needs Package A) or database lock cycles.

Pathfinding & Backtracking Solutions:

- Ideal for exhaustive space search, maze solving, puzzle solving (N-Queens, Sudoku), and game tree evaluation.
- *Example:* **Navigating a Physical Maze or N-Queens Puzzle.** Walking down a corridor until you hit a dead wall, then stepping back to the last intersection to try another turn.